



REENIDHI
EDUCATIONAL GROUP

REENIDHI
INSTITUTE OF
SCIENCE AND
TECHNOLOGY



COURSE

ARTIFICIAL INTELLIGENCE AND DEEP LEARNING (AIDL)

UNIT-IV

Mr. P Ramu, Assistant Professor

UNIT IV

Uncertain Knowledge and Reasoning :

1. Bayes' Rule,
2. Concepts of Time and Uncertainty,
3. Utility Functions,
4. Value of Information,
5. Value iteration,
6. Policy iteration,
7. Partially Observable MDP

Uncertain Knowledge and Reasoning

Uncertain Knowledge and Reasoning in AI is a core concept in Artificial Intelligence that deals with making decisions when information is incomplete, noisy, or ambiguous—just like real-world situations.

What is Uncertainty in AI?

Uncertainty arises when:

1. Data is **incomplete** (missing values)
2. Observations are **noisy or imprecise**
3. The environment is **dynamic or unpredictable**

Example: A weather prediction system cannot say with 100% certainty that it will rain tomorrow.

What is Reasoning in AI?

- Reasoning refers to:
Using logic, rules, and knowledge to infer new information from existing data.

Example:

- If “All humans are mortal” and “Ram is a human.”
- Then AI concludes $\rightarrow$ “Ram is mortal.”

Methods for Handling Uncertainty

1. Probability Theory.
2. Bayesian Networks.
3. Fuzzy Logic.
4. Markov Models.
5. Utility Theory.

Methods for Handling Reasoning

1. Logical Reasoning.
2. Rule-Based Reasoning
3. Probabilistic Reasoning.
4. Fuzzy Logic Reasoning.
5. Non-Monotonic Reasoning.
6. Case-Based Reasoning (CBR).
7. Model-Based Reasoning.

1. Bayes' Rule

Bayes' Theorem is a mathematical formula used to determine the conditional probability of an event based on prior knowledge and new evidence.

It adjusts probabilities when new information comes in and helps make better decisions in uncertain situations.

Bayes' rule follows three conditions.

1. Conditional probability.
2. Independent events.
3. Dependent events.

1. Conditional probability

- **Problem**

- A card is drawn from a deck.

Given that the card is a **face card**, what is the probability it is a **King**?

- **Solution**

- Let:

- A = card is King

- B = card is a face card

- We know:

- Total face cards = 12 (J, Q, K of 4 suits)

- Kings = 4

- Using conditional probability:

- $$P(A | B) = \frac{P(A \cap B)}{P(B)} = \frac{4}{12} = \frac{1}{3}$$

2. Independent Events Example

Problem

A coin is tossed and a dice is rolled.
Find the probability of getting:
Head and a 4

Solution

- Let:
- $A = \text{Head} \rightarrow P(A) = \frac{1}{2}$
- $B = \text{Getting 4} \rightarrow P(B) = \frac{1}{6}$
- Since they are independent:
- $P(A \cap B) = P(A) \cdot P(B) = \frac{1}{2} \times \frac{1}{6} = \frac{1}{12}$

3. Dependent Events Example

Problem

A bag has 5 red and 3 blue balls.
Two balls are drawn **without replacement**.
Find probability both are red.

Solution

Let:

First red: $P(R_1) = \frac{5}{8}$

Second red (after one red removed): $P(R_2 | R_1) = \frac{4}{7}$

Since dependent:

- $$P(R_1 \cap R_2) = \frac{5}{8} \times \frac{4}{7} = \frac{20}{56} = \frac{5}{14}$$

$$P(A|B) = \frac{P(B|A) P(A)}{P(B)}$$

Meaning of Terms

- $P(A)$ → Prior probability (initial belief)
- $P(B | A)$ → Likelihood (evidence given hypothesis)
- $P(B)$ → Evidence (total probability of B)
- $P(A | B)$ → Posterior probability (updated belief)

Bayes' Rule

we have,

$$P(A|B) = \frac{P(A \cap B)}{P(B)}$$

$$P(A \cap B) = P(A|B) P(B)$$

$$P(A \cap B) = P(B|A) P(A)$$

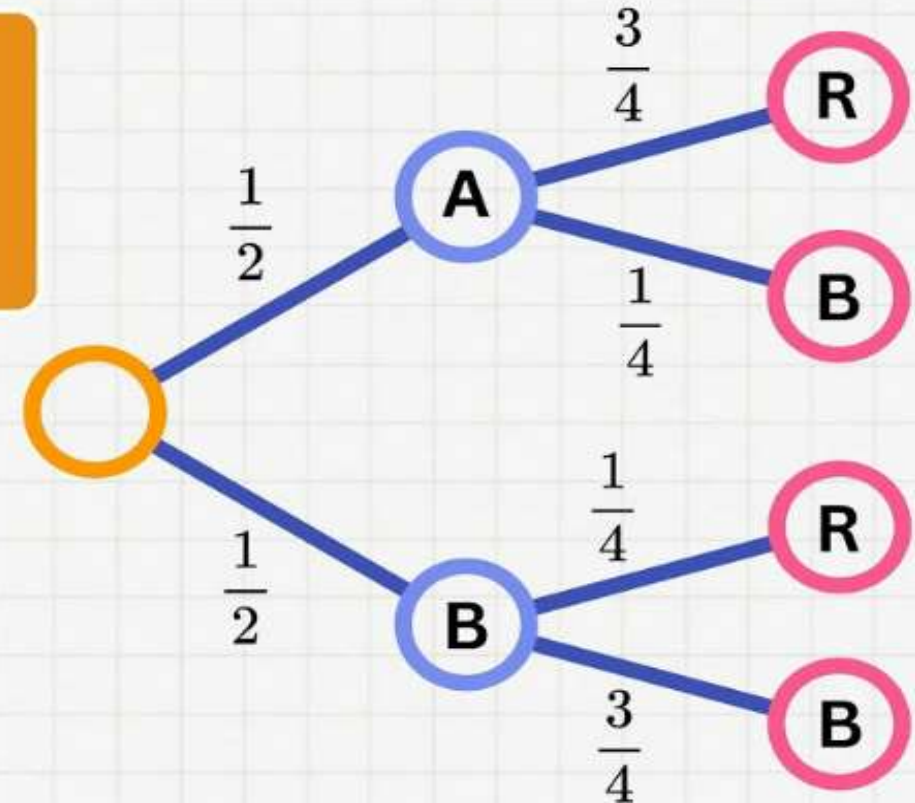
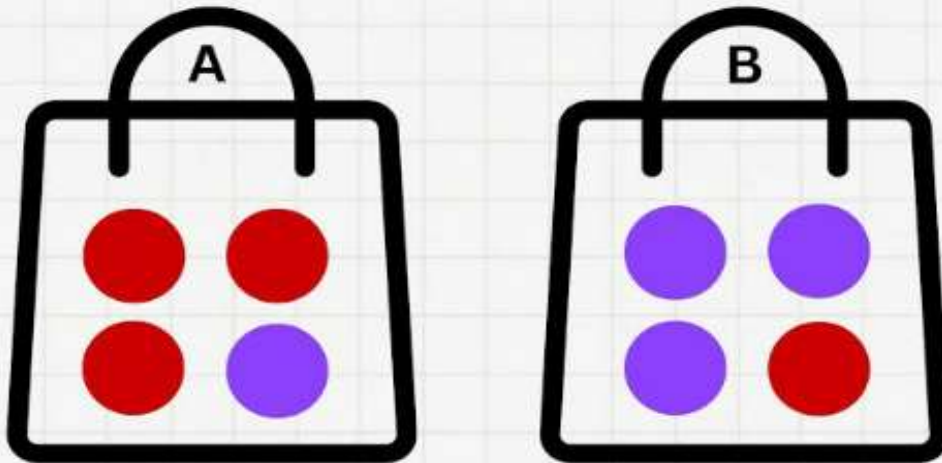
$$P(A|B) P(B) = P(B|A) P(A)$$

$$P(A|B) = \frac{P(B|A) P(A)}{P(B)}$$

(Bayes' Rule)

Bayes Theorem Explained

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)}$$



Problem: Sreenidhi has two bags. Bag I has 7 red and 4 blue balls, and Bag II has 5 red and 9 blue balls. Sreenidhi draws a ball at random, and it turns out to be red. Determine the probability that the ball was from the bag I.

- **Given Data**
- **Bag I:** 7 red, 4 blue $\rightarrow$ total = 11
- **Bag II:** 5 red, 9 blue $\rightarrow$ total = 14
- Assume a bag is chosen **randomly**:
- $P(B_1) = P(B_2) = \frac{1}{2}$

Let: R = event of drawing a red ball

Step 1: Conditional Probabilities

$$P(R | B_1) = \frac{7}{11}, P(R | B_2) = \frac{5}{14}$$

- **Step 2: Total Probability of Red**

$$\begin{aligned}P(R) &= P(R | B_1)P(B_1) + P(R | B_2)P(B_2) \\&= \frac{7}{11} \cdot \frac{1}{2} + \frac{5}{14} \cdot \frac{1}{2} \\&= \frac{7}{22} + \frac{5}{28} = \frac{98 + 55}{308} = \frac{153}{308}\end{aligned}$$

Step 3: Apply Bayes' Rule

$$\begin{aligned}P(B_1 | R) &= \frac{P(R | B_1)P(B_1)}{P(R)} \\&= \frac{\frac{7}{11} \cdot \frac{1}{2}}{\frac{153}{308}} = \frac{7}{22} \times \frac{308}{153} \\&= \frac{98}{153}\end{aligned}$$

2. Concepts of Time and Uncertainty

AI systems **deal with changing environments (time) and incomplete or uncertain information (uncertainty)**—both are essential for real-world decision-making.

1. Concept of Time in AI

What is Time?

- Time represents how a system **changes from one state to another.**

AI must:

- Remember past states
- Analyze present state
- Predict future states

Types of Time Representation

1. Discrete Time

Time is divided into steps $t = 0, 1, 2, 3, \dots$

Example: Daily stock prices

2. Continuous Time

Time flows continuously

Example: Temperature monitoring

Temporal Relationships

Describes how events relate over time:

1. Before .
2. After .
3. During .
4. Simultaneous.

Temporal Models

State-Space Representation

- System moves from one state to another over time

Markov Models

- Future depends only on present state

Includes:

- Markov Decision Process
- Hidden Markov Models (HMM)

Temporal Logic

- Logic with time-based expressions

Example: “Eventually the system stops”

2. Concept of Uncertainty in AI

What is Uncertainty?

Uncertainty arises when:

1. Data is incomplete
2. Observations are noisy
3. The future is unpredictable

Types of Uncertainty

1. Aleatory Uncertainty

- Due to randomness

Example: Tossing a coin

2. Epistemic Uncertainty

Due to a lack of knowledge

Example: Incomplete medical diagnosis

Methods to Handle Uncertainty

1. Probability Theory

Uses numerical probabilities

Key concept:

Bayes' Theorem

2. Fuzzy Logic

Handles vague information

Example: “Temperature is hot”

3. Bayesian Networks

Graph-based probabilistic models

4. Utility Theory

Helps in decision-making

3. Combining Time and Uncertainty

Real-world AI problems involve both!

Example:

Weather prediction:

- Changes over time
- Uncertain outcomes

Models Combining Both

1. Hidden Markov Models (HMM)

Hidden states + observable outputs.

2. Dynamic Bayesian Networks

Extend Bayesian models over time.

3. Markov Decision Process (MDP)

Decision-making over time under uncertainty.

Applications

1. Speech recognition 
2. Robotics 
3. Autonomous vehicles 
4. Stock market prediction 
5. Medical diagnosis 

Key Differences

Aspect	Time Concept	Uncertainty Concept
Focus	Change over time	Lack of certainty
Example	Weather over days	Probability of rain
Models	MDP, HMM	Bayes, Fuzzy Logic

3. Utility Function in Artificial Intelligence

A **utility function** is a mathematical tool that helps an AI system **measure how good or desirable an outcome is** and choose the best action.

Definition

A utility function assigns a numerical value to each outcome:

- $U(x)$
- x =outcome
- $U(x)$ =utility (preference value)
- ✓ Higher value $\rightarrow$ better outcome
- ✓ Lower value $\rightarrow$ worse outcome

Why Utility Functions are Important

In real-world AI:

- Outcomes are uncertain
- Multiple actions are possible

Utility functions help to:

Compare different choices

- Handle uncertainty
- Make optimal decisions

Expected Utility (Decision Rule)

- AI selects the action with **maximum expected utility**:
- $EU(A) = \sum P(o_i | A) U(o_i)$

simple Example

An AI chooses between two actions:

- **Action A:**

- 80% success $\rightarrow$ Utility = 100

20% failure $\rightarrow$ Utility = 0

$$EU(A) = 0.8 \times 100 = 80$$

Action B:

60% success $\rightarrow$ Utility = 150

40% failure $\rightarrow$ Utility = 0

$$EU(B) = 0.6 \times 150 = 90$$

Decision:

Choose **Action B** (higher expected utility)

Types of Utility Functions: Three Utility Functions

1. Risk-Averse

Prefers safer outcomes

Utility increases slowly (concave)

2. Risk-Neutral

Considers only expected value

Linear utility

3. Risk-Seeking

Prefers risky outcomes

Utility increases rapidly (convex)

Properties

1. Represents preferences
2. Helps in rational decision-making
3. Used with probability

Applications

- Medical diagnosis 
- Robotics 
- Autonomous vehicles 
- Game playing 
- Economics 

UTILITY FUNCTIONS

A utility function $U(x)$ assigns a real number to each outcome x to represent how desirable that outcome is.

Higher utility $\Rightarrow$ more preferred outcome

UTILITY FUNCTION

$U(x)$ = utility of outcome x

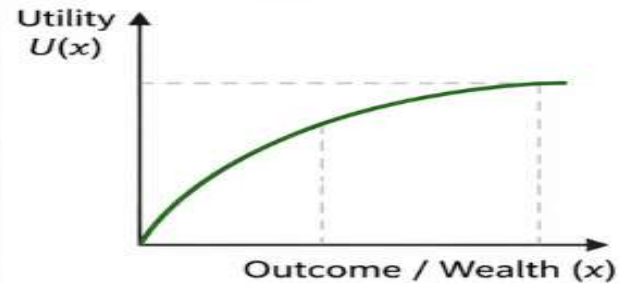


EXPECTED UTILITY

$$EU(A) = \sum P(o_i|A) \times U(o_i)$$

Choose the action with maximum expected utility.

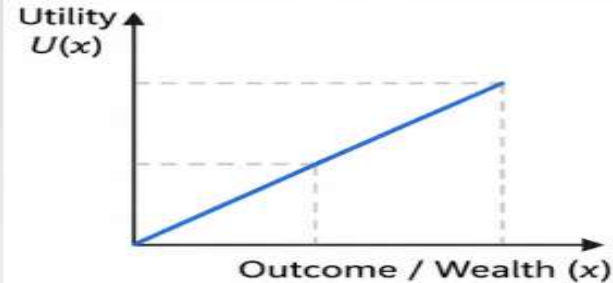
1. RISK-AVERSE (Concave)



Increasing, at a decreasing rate

- Prefers certain outcomes
- Avoids risk
- Example: Fixed salary

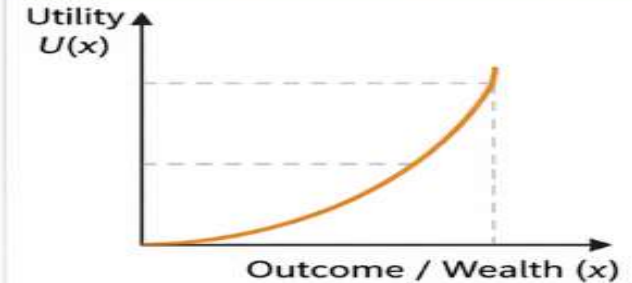
2. RISK-NEUTRAL (Linear)



Increasing, at a constant rate

- Indifferent to risk
- Only expected value matters
- Example: Business decisions

3. RISK-SEEKING (Convex)



Increasing, at an increasing rate

- Prefers risky outcomes
- Seeks risk
- Example: Gambling

Key Idea: The shape of the utility function represents the decision maker's attitude toward risk.
AI chooses actions that maximize expected utility.

4. Value of Information (VoI)

Introduction

- In Artificial Intelligence, decision-making often occurs under uncertainty. The **Value of Information (VoI)** is a concept that helps determine **whether acquiring additional information will improve the quality of a decision.**

Definition

Value of Information is the increase in expected utility obtained by acquiring additional information before making a decision.

- **Formula**

$$\text{VoI} = \text{EU}(\text{with information}) - \text{EU}(\text{without information})$$

Where: EU= Expected Utility.

Types of Value of Information

1. Value of Perfect Information (VPI)

- Assumes **completely accurate information**
- Provides the **maximum possible improvement**

2. Value of Imperfect Information

- Information may be **uncertain or noisy**
- More realistic in practical scenarios

Steps to Calculate VoI

- Compute expected utility **without information**
- Compute expected utility **with information**
- Subtract:
- $VoI = EU_{with} - EU_{without}$

Example

A doctor must choose a treatment:

Without Test:

- Treatment A $\rightarrow$ EU = 80
- Treatment B $\rightarrow$ EU = 70
- $\hookrightarrow$ Best choice = A $\rightarrow EU_{without} = 80$

With Test:

- After performing a test $\rightarrow$ better decision
- Expected Utility = 90

Calculation:

- $VoI = 90 - 80 = 10$

The information improves decision quality by **10 units**

Decision Rule

- If **VoI** > **Cost of information** $\rightarrow$ Acquire information ✓
- If **VoI** $\leq$ **Cost** $\rightarrow$ Do not acquire ✗

Applications

- Medical diagnosis (lab tests)
- Business and investment decisions
- Robotics and sensor systems
- Weather prediction
- Risk analysis

Advantages

- ✓ Improves decision accuracy
- ✓ Helps avoid unnecessary data collection
- ✓ Saves cost and time

5. Value Iteration

- **Value Iteration** is a fundamental algorithm used to solve decision-making problems modeled as a Markov Decision Process. It helps an agent find the **optimal policy** by computing the best value for each state.

Introduction

In many real-world problems:

- The outcome of actions is uncertain
- Decisions must be made over time

Value Iteration is used to compute **optimal decisions step-by-step**.

Bellman Optimality Equation

$$V^*(s) = \max_a \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma V^*(s')]$$

Explanation:

- $V^*(s)$: Optimal value of state s
- a : Action
- $P(s'|s, a)$: Probability of moving to next state s'
- $R(s, a, s')$: Reward received
- γ : Discount factor ($0 \leq \gamma \leq 1$)

Algorithm Steps

1. Initialize $V(s) = 0$ for all states
2. Repeat:
 - Update each state using Bellman equation
3. Stop when values **converge**
4. Extract optimal policy

Simple Example

- Suppose:
- Two states: S_1, S_2
- Rewards:
 - $S_1 \rightarrow +10$
 - $S_2 \rightarrow +5$

Iteration 1:

- $V(S_1) = 10, V(S_2) = 5$
- **Iteration 2:**
- Values updated using future rewards
- ➞ Continue until stable values are reached

Policy Extraction

- After computing values:
- $\pi(s) = \arg \max_a \sum_{s'} P(s' | sa) [R + \gamma V(s')]$
- ➞ Gives **best action for each state**

- **Bellman Equation**
- $V(s) = R(s) + \gamma V(s')$

Step 1: Initialize Values

- $V_0(S_1) = 0, V_0(S_2) = 0$

Step 2: Iteration 1

- $V_1(S_1) = 5 + 0.9 \times 0 = 5$
- $V_1(S_2) = 2 + 0.9 \times 0 = 2$

Step 3: Iteration 2

- $V_2(S_1) = 5 + 0.9 \times V_1(S_2) = 5 + 0.9 \times 2 = 6.8$
- $V_2(S_2) = 2 + 0.9 \times V_1(S_2) = 2 + 0.9 \times 2 = 3.8$

Step 4: Iteration 3

- $V_3(S_1) = 5 + 0.9 \times 3.8 = 8.42$
- $V_3(S_2) = 2 + 0.9 \times 3.8 = 5.42$

Step 5: Iteration 4

- $V_4(S_1) = 5 + 0.9 \times 5.42 = 9.878$
- $V_4(S_2) = 2 + 0.9 \times 5.42 = 6.878$

Advantages

- ✓ Guarantees optimal solution
- ✓ Simple and widely used
- ✓ Works well with known models

Limitations

- ✗ Computationally expensive for large states
- ✗ Requires full model (probabilities, rewards)

Applications

- Robotics 
- Game playing 
- Autonomous vehicles 
- Resource management

6. Policy Iteration

Policy Iteration is an algorithm used to find the **optimal policy** (best action for each state) in a Markov Decision Process.

Introduction

In decision-making problems:

- An agent must choose actions in different states
- Each action gives rewards over time

Policy Iteration finds the **best strategy (policy)** by:

- Evaluating a policy
- Improving it

Two Main Steps

1. Policy Evaluation

- Compute value of each state for a given policy
- $V^\pi(s) = \sum_{s'} P(s' | s, \pi(s)) [R(s, \pi(s), s') + \gamma V^\pi(s')]$

2. Policy Improvement

- Update policy by choosing better actions
- $\pi'(s) = \arg \max_a \sum_{s'} P(s' | s, a) [R(s, a, s') + \gamma V(s')]$

• Algorithm Steps

1. Initialize a random policy π

2. Policy Evaluation:

- Compute $V^\pi(s)$ for all states

3. Policy Improvement:

- Update policy using max operator

4. Repeat steps 2 & 3 until policy stops changing

Simple Example

States:

- S_1, S_2

Actions:

- a_1, a_2

Step 1: Initial Policy

- $\pi(S_1) = a_1$
- $\pi(S_2) = a_2$

Step 2: Policy Evaluation

Compute values based on current policy

Step 3: Policy Improvement

Check if better actions exist:

- If yes $\rightarrow$ update policy
- If no $\rightarrow$ optimal policy found

Advantages

- ✓ Faster convergence in many cases
- ✓ Directly gives optimal policy
- ✓ Easy to understand conceptually

Limitations

- ✗ Policy evaluation can be costly
- ✗ Requires full knowledge of model

Applications

- Robotics 
- Game AI 
- Autonomous vehicles 
- Resource allocation.

step-by-step numerical example of Policy Iteration

Problem Setup

States:

- S_1, S_2

Actions:

- a_1, a_2
- Discount factor:
- $\gamma = 0.9$

Transition & Rewards

- **From S_1 :**
- $a_1 \rightarrow S_1$, Reward = 5
- $a_2 \rightarrow S_2$, Reward = 10
- **From S_2 :**
- $a_1 \rightarrow S_1$, Reward = 2
- $a_2 \rightarrow S_2$, Reward = 3

Step 1: Initialize Policy

Assume initial policy:

$$\pi_0(S_1) = a_1, \pi_0(S_2) = a_2$$

Step 2: Policy Evaluation

Using current policy:

For S_1 :

$$\begin{aligned} V(S_1) &= 5 + 0.9V(S_1) \\ V(S_1)(1 - 0.9) &= 5 \Rightarrow V(S_1) = 50 \end{aligned}$$

For S_2 :

$$\begin{aligned} V(S_2) &= 3 + 0.9V(S_2) \\ V(S_2)(1 - 0.9) &= 3 \Rightarrow V(S_2) = 30 \end{aligned}$$

Step 3: Policy Improvement

Now check if better actions exist.

For S_1 :

$$\begin{aligned} a_1: 5 + 0.9(50) &= 50 \\ a_2: 10 + 0.9(30) &= 37 \end{aligned}$$

Best action = (a_1 no change)

For S_2 :

$$\begin{aligned} a_1: 2 + 0.9(50) &= 47 \\ a_2: 3 + 0.9(30) &= 30 \end{aligned}$$

Best action = (a_1 change policy)

- **Updated Policy**
- $\pi_1(S_1) = a_1, \pi_1(S_2) = a_1$

Step 4: Policy Evaluation (New Policy)

- **For S_2 :**
- $V(S_2) = 2 + 0.9V(S_1)$
- **For S_1 :**
- $V(S_1) = 5 + 0.9V(S_1) \Rightarrow V(S_1) = 50$
- Now substitute:
- $V(S_2) = 2 + 0.9(50) = 47$

Step 5: Policy Improvement Again

- **For S_1 :**
 - $a_1: 50$
 - $a_2: 10 + 0.9(47) = 52.3$
 - $\hookrightarrow$ Choose a_2 (update)
 - **For S_2 :**
 - $a_1: 47$
 - $a_2: 3 + 0.9(47) = 45.3$
- Choose a_1 (no change)

- **Updated Policy**
- $\pi_2(S_1) = a_2, \pi_2(S_2) = a_1$

Step 6: Final Check

- Re-evaluate and improve:
No further changes → **Optimal policy reached**

Final Answer

- **Optimal Policy:**
- $S_1 \rightarrow a_2$
- $S_2 \rightarrow a_1$
- **State Values:**
- $V(S_1) \approx 52.3$
- $V(S_2) \approx 47$

7. Partially Observable MDP

- A **Partially Observable Markov Decision Process (POMDP)** is an extension of a Markov Decision Process where the agent **cannot directly observe the true state of the environment**.

Introduction

- In many real-world situations:
- The system state is **hidden or uncertain**
- The agent receives only **partial or noisy observations**

POMDP helps an agent **make optimal decisions under uncertainty and incomplete information**

Components of POMDP

A POMDP is defined as:

$$(S, A, P, R, O, Z, \gamma)$$

Where:

- S = Set of states
- A = Set of actions
- $P(s' | sa)$ = Transition probability
- $R(s, a)$ = Reward function
- O = Set of observations
- $Z(o | s'a)$ = Observation probability
- γ = Discount factor

Belief State

- Since the true state is unknown, the agent uses:
- **Belief state** $b(s)$ =probability that system is in state s
- Belief is updated after every action and observation

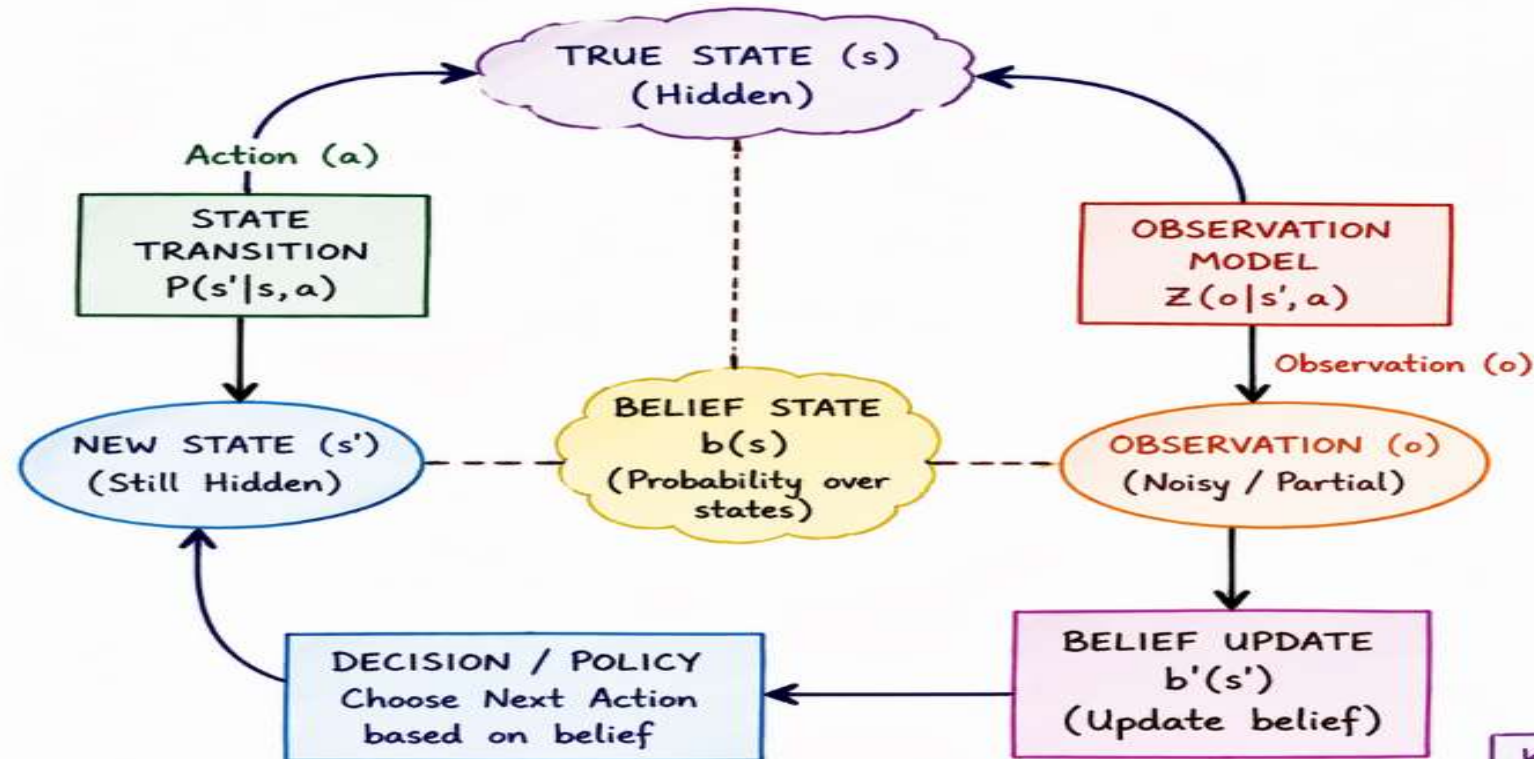
Belief Update Equation

- $b'(s') = \eta Z(o' | s' a) \sum_s P(s' | sa) b(s)$

Where:

- η = normalization constant
- $b'(s')$ =updated belief

POMDP DIAGRAM



Components

S : set of states
 A : set of actions
 $P(s'|s, a)$: Transition
 $R(s, a)$: Reward
 O : set of observations
 $Z(o|s', a)$: Observation
 γ : Discount factor

Key Idea:

Since the true state is hidden, the agent maintains a belief over states and updates it using observations.

How it works:

Action $\rightarrow$ State $\rightarrow$ Observation $\rightarrow$ Belief Update $\rightarrow$ Action
 (Repeat)

Example Problem: POMDP (Belief Update)

- **◆ Problem**
- A robot can be in two states:
- S_1 =Clean room
- S_2 =Dirty room
- Initial belief:
- $b(S_1) = 0.6, b(S_2) = 0.4$
- Action: Robot performs “**Clean**”
- Transition model:
- $P(S_1' | S_1 \text{Clean}) = 0.9$
- $P(S_1' | S_2 \text{Clean}) = 0.7$
- Observation: Robot senses “**Clean**”
- Observation model:
- $Z(\text{Clean} | S_1) = 0.8$
- $Z(\text{Clean} | S_2) = 0.3$
-  Find the **updated belief** $b'(S_1)$

Step 1: Prediction (Before Observation)

$$\begin{aligned}\hat{b}(S_1) &= P(S_1 | S_1)b(S_1) + P(S_1 | S_2)b(S_2) \\ &= (0.9 \times 0.6) + (0.7 \times 0.4) = 0.54 + 0.28 = 0.82\end{aligned}$$

Step 2: Apply Observation

$$\begin{aligned}b'(S_1) &\propto Z(\text{Clean} | S_1) \cdot \hat{b}(S_1) \\ &= 0.8 \times 0.82 = 0.656\end{aligned}$$

Step 3: Normalize

Similarly for S_2 :

$$\begin{aligned}\hat{b}(S_2) &= 1 - 0.82 = 0.18 \\ b'(S_2) &\propto 0.3 \times 0.18 = 0.054\end{aligned}$$

Total:

$$0.656 + 0.054 = 0.71$$

•

Step 4: Final Beliefs

$$\begin{aligned}b'(S_1) &= \frac{0.656}{0.71} \approx 0.924 \\ b'(S_2) &= \frac{0.054}{0.71} \approx 0.076\end{aligned}$$

Final Answer:

$$\hookrightarrow b'(S_1) \approx 0.924$$

$$\hookrightarrow b'(S_2) \approx 0.076$$